Project Report: Student Result Management System

Track Name: Python GUI

Developer:

1. Mahin Sarker (Id: 22-48338-3)
2. DIpannita Pramanik (Id: 22-48286-3)

1. Project Overview:

The Student Result Management System is a modern, offline desktop application built using Python. It provides educational administrators with a clean, intuitive interface to seamlessly manage student records, course details, and academic results. By replacing manual paperwork with automated grading and instant statistical reporting, this application streamlines the entire evaluation process.

2. System Architecture & Tech Stack:

The Student Result Management System is an offline, standalone desktop application designed to digitize and automate the academic grading process.

- **Frontend GUI:** Built using Python's customtkinter to provide a modern, responsive, and dark/light-mode compatible interface. tkinter.ttk is utilized for rendering robust, scrollable data tables (Treeviews).
- **Backend Storage:** Operates on a lightweight, serverless CSV-based database architecture (users.csv, courses.csv, students.csv, results.csv).
- **Design Pattern:** The system uses a modular Model-View-Controller (MVC) approach, separating the UI views from the underlying data manager classes.

Libraries used:

- customtkinter (Modern UI Layer)
- tkinter.ttk (Data Grid/Treeview)
- CSV (Database Handling)
- messagebox (System Alerts)

3. Detailed Module Breakdown:

Module 1: User Authentication & Security (registration_view.py & main.py)

This module handles secure access to the application.

- **Account Creation:** Users can register by providing their Full Name, Phone, Username, and Password.
- **Input Validation:** The system enforces strict validation rules (e.g., passwords must be at least 8 characters, phone numbers must be exactly 11 digits).
- **Duplicate Prevention:** The database is scanned during registration to ensure no two accounts share the same username.
- **Login System:** Authenticates users against the users.csv database before granting access to the main dashboard.

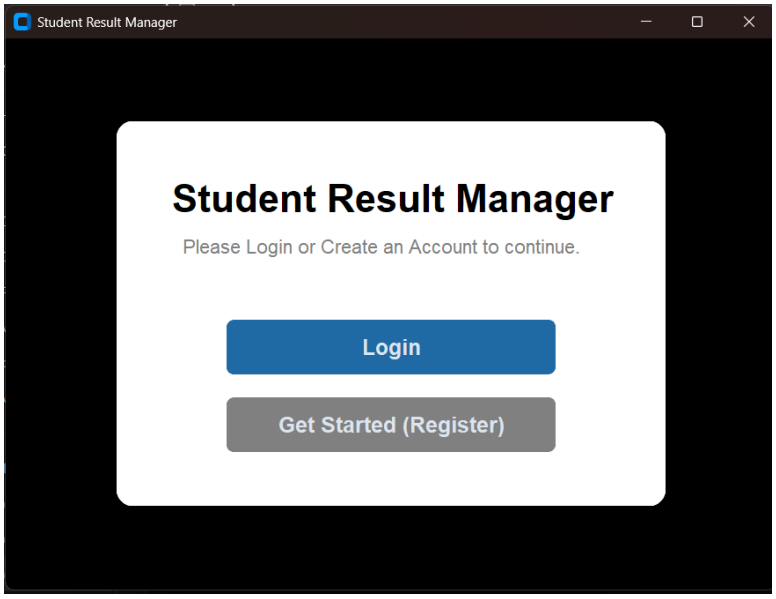

Figure-1: main.py

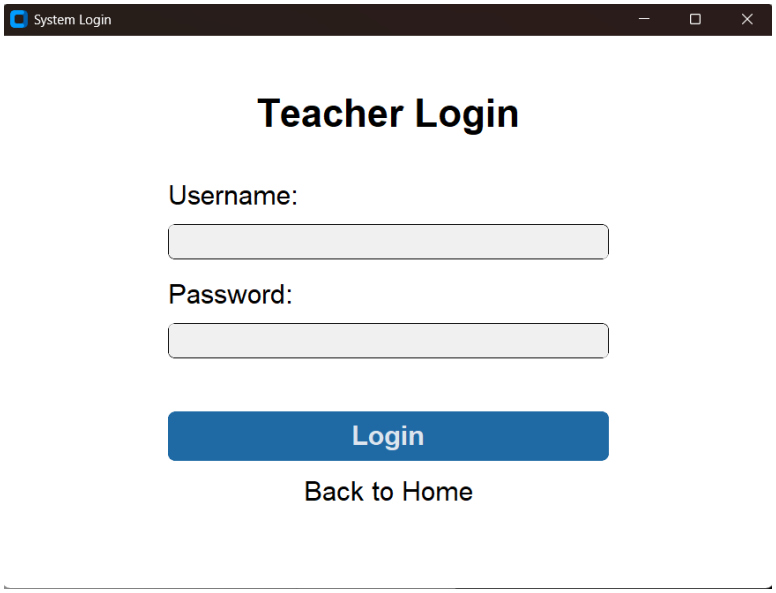

Figure-2: login.py

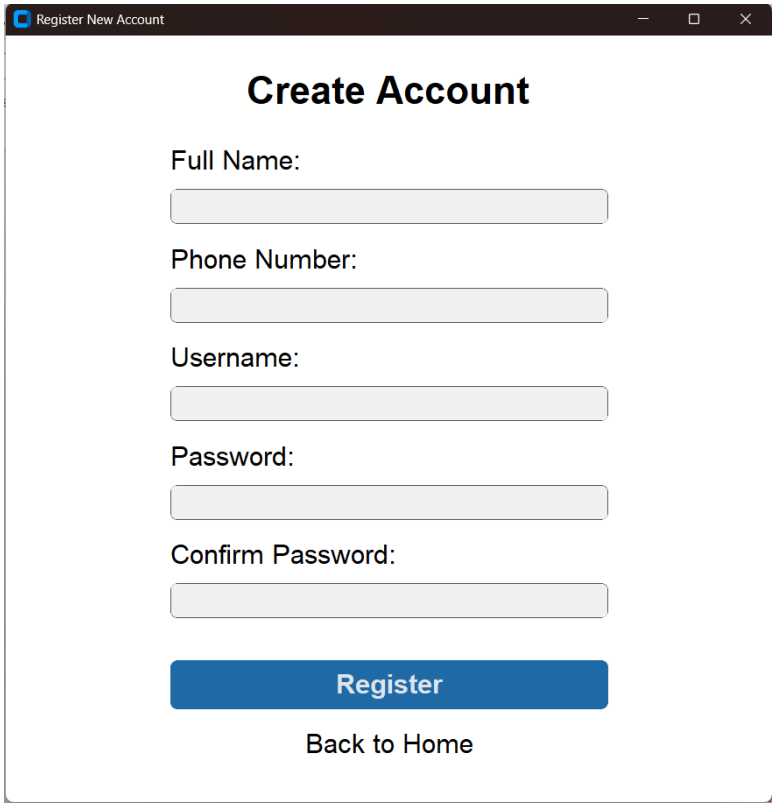

Figure-3: registration.py

Module 2: Main Dashboard & Navigation (dashboard.py)

The central hub of the application.

- **Dynamic Frame Switching:** Utilizes a persistent sidebar menu allowing users to seamlessly switch between Courses, Students, Results, and Statistics pages without opening new windows.
- **Manager Injection:** Initializes the CourseManager(), StudentManager(), and ResultManager() at startup and passes them to the respective UI frames, ensuring all pages interact with the same live data.
- **Safe Logout:** Properly destroys the current session and re-launches the authentication screen.

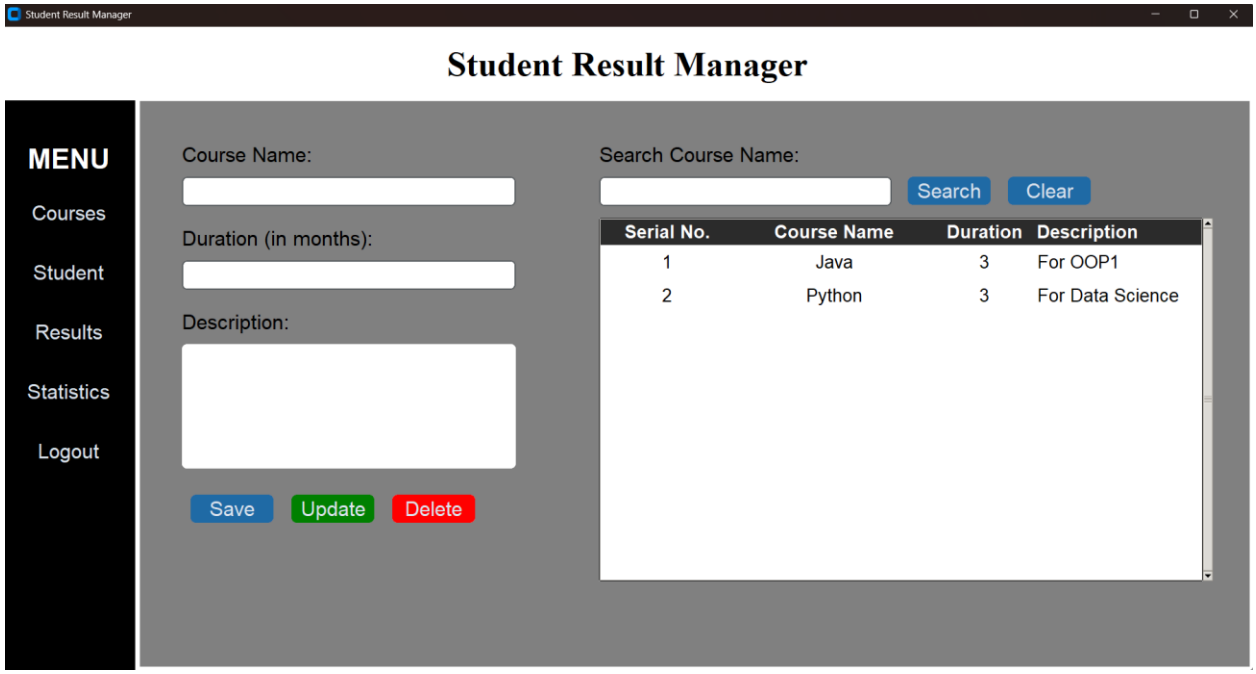

Figure-4: dashboard.py

Module 3: Course Management (course_view.py)

Allow an administrator to define the academic curriculum.

- **CRUD Operations:** Full capability to Add, Update, and Delete courses.
- **Smart Serial IDs:** Automatically generates and permanently assigns a unique "Serial No." to new courses so they can be tracked even if other courses are deleted.
- **Validation Rules:** Ensures course durations do not exceed 12 months and prevents duplicate course names.
- **Search Functionality:** Includes a search bar to instantly filter the course table by name.

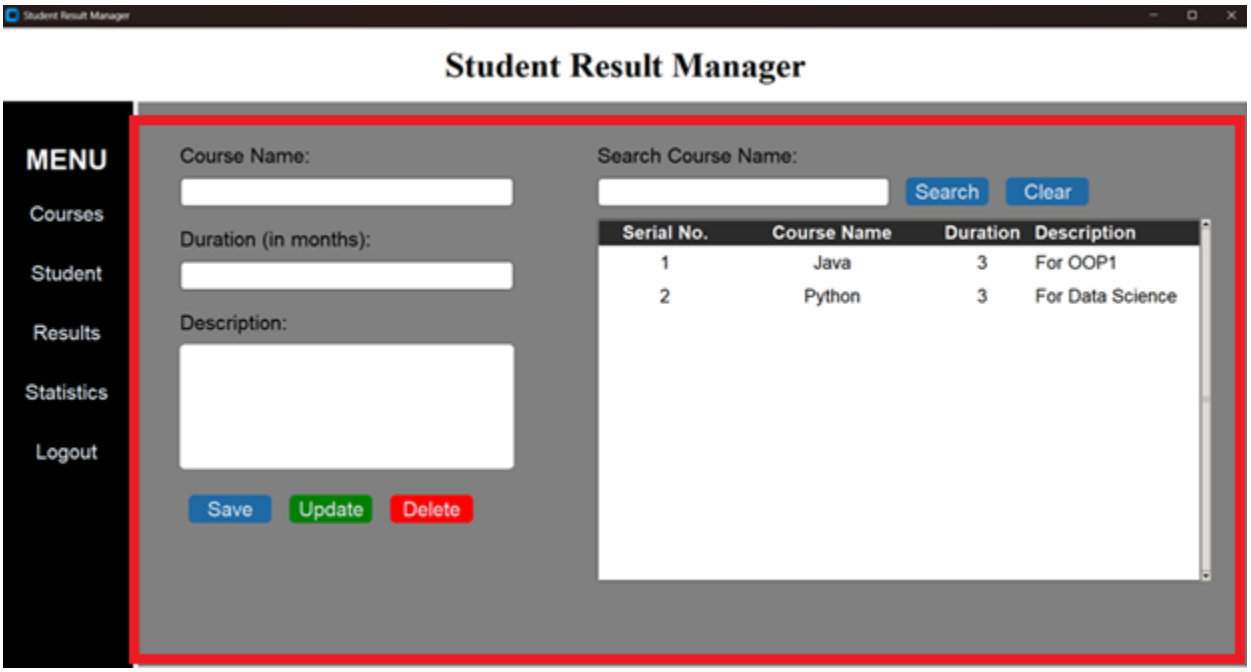

Figure-5: course_view.py

Module 4: Student Registration (student_view.py)

Manages the core demographic data of the student body.

- **Data Tracking:** Captures Roll Number, Name, Phone, Gender (via dropdown), and Address.
- **Strict Primary Keys:** Enforces a rigid duplicate check on "Roll Numbers." The system completely blocks the creation or updating of a student if the Roll Number is already in use by another student.
- **Data Formatting:** Ensures Roll Numbers and Phone Numbers contain only valid numeric digits.

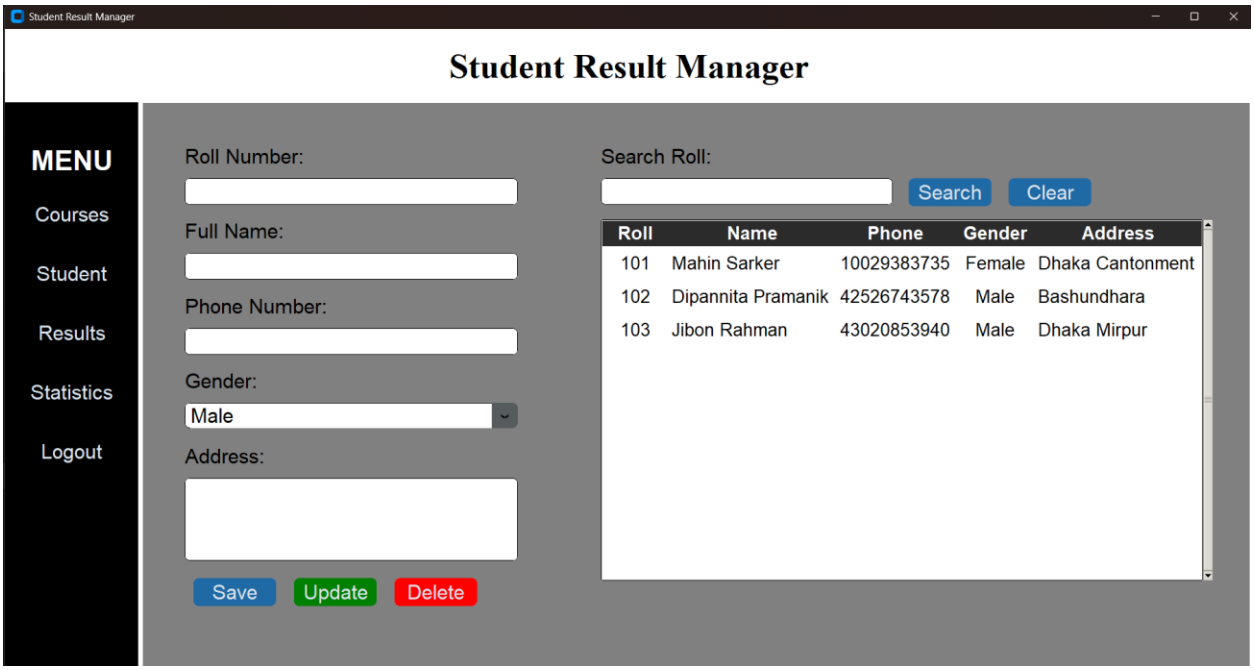

Figure-6: student_view.py

Module 5: Result Processing & Auto-Grading (result_view.py)

The core academic engine of the application.

- **Relational Dropdowns:** Automatically pulls live data from the Student and Course databases, allowing the teacher to select a student's Roll Number and a Course Name from dropdown menus rather than typing them manually.
- **Automated Grade Calculation:** Teachers only input raw marks (0-100). The system's internal logic automatically calculates the corresponding letter grade (A+, A, B, F, etc.) based on predefined academic brackets.
- **Duplicate Grade Prevention:** The system scans the database to ensure a student cannot be assigned two different grades for the exact same course.
- **True Indexing Logic:** Implements an advanced hidden indexing system in the data table. This ensures that if a user searches for a specific record and clicks "Delete" or "Update", the system modifies the correct record in the backend database, preventing accidental data loss.

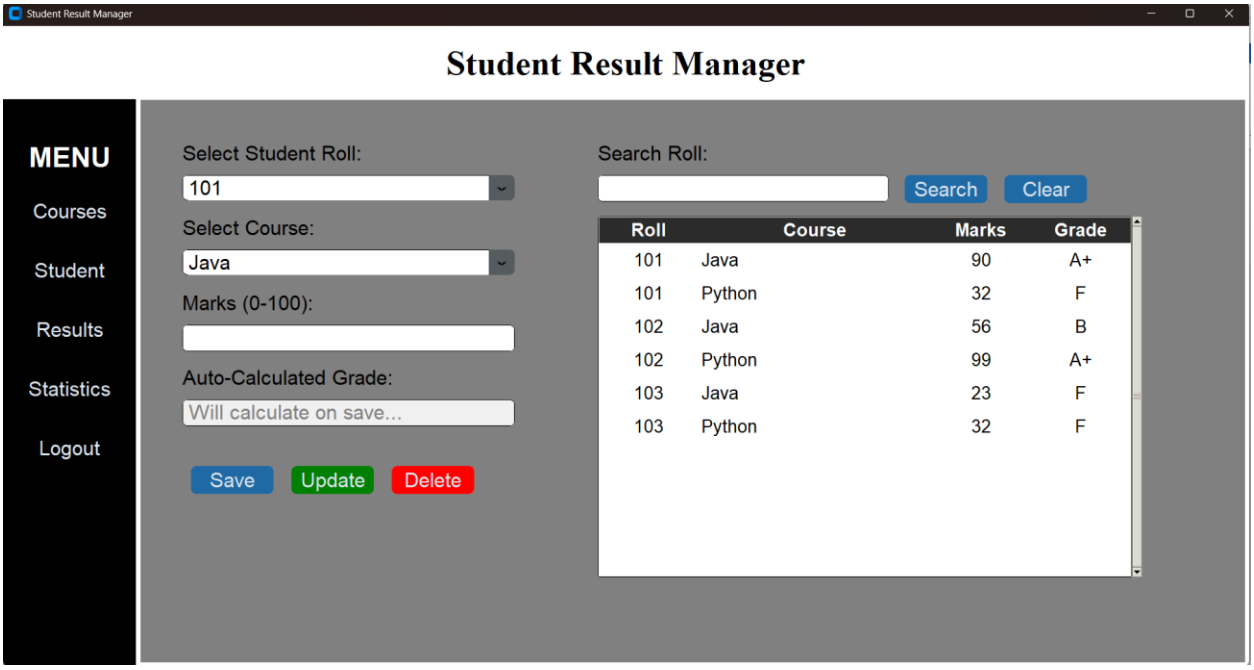

Figure-7: result_view.py

Module 6: Analytics & Reporting (overall_report_view.py)

Provides a high-level, visual overview of class performance.

- **Live Statistics:** Dynamically reads the database to display Total Registered Students and the Total Number of Evaluated Students.
- **Overall Pass/Fail Logic:** Processes every student's complete academic record. If a student receives an "F" in *any* course, the system automatically classifies them as "Failed" overall.
- **Visual Status Table:** Generates a comprehensive summary table showing every evaluated student's final status.
- **Color-Coded UI:** Utilizes visual tags to automatically color the table rows (Light Green for PASSED, Light Red for FAILED), allowing administrators to identify struggling students briefly.

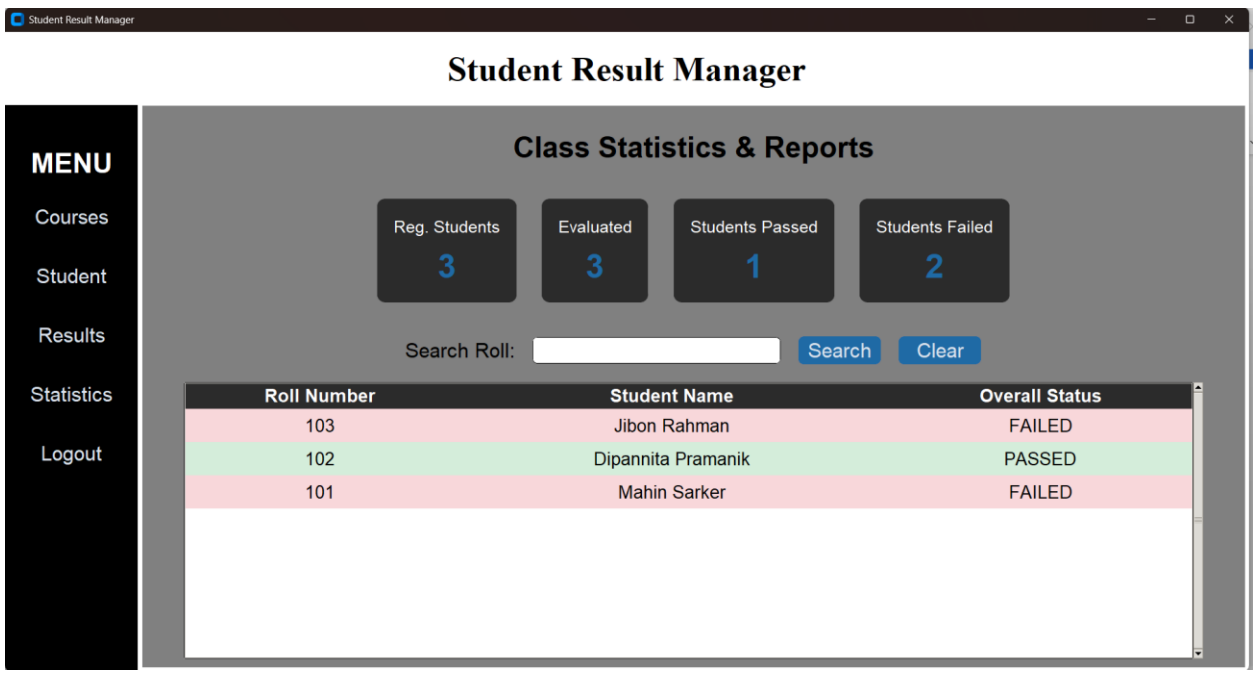

Figure-8: overall_report_view.py

4. Object-Oriented Programming (OOP):

The Student Result Management System is engineered using Modular OOP Design. This architecture ensures that the data (Models), the operations (Managers), and the interface (Views) are decoupled, making the system scalable and easy to maintain.

Key OOP Principles Applied:

A. Class-Based Modeling (Encapsulation)

The system defines clear Blueprints (Classes) for every real-world entity.

- **Data Models:** Classes like Student, Course, and Result encapsulate all attributes of a record into a single object.
- **Logic Managers:** Classes like StudentManager and ResultManager encapsulate the "Business Logic" (CRUD operations), keeping the database interaction separate from the visual interface.

B. Inheritance (UI Component Reusability)

The application leverages **Inheritance** by extending the existing customtkinter library.

- Each major page (e.g., StudentView, CourseView) inherits from ct.CTkFrame. This allows the views to behave like standard UI containers while hosting custom logic, tables, and validation rules unique to that module.

C. Separation of Concerns (Abstraction)

By using a multi-layer approach, the project achieves high levels of abstraction:

1. **Storage Layer:** CSVStorage handles the "low-level" file I/O (reading/writing rows).
2. **Manager Layer:** Bridges between data and UI, handling search logic and duplicates.
3. **View Layer:** Focuses purely on user interaction and data presentation.

D. Method Polymorphism & Uniformity

The system maintains a consistent naming convention across different modules (e.g., to_dict(), setup_ui(), clear_entries()). This uniformity makes the codebase predictable and allows for easier debugging and future feature expansion.

5. System Workflow:

User Login/Register → Dashboard Sidebar → Select Module → Input/Modify Data → Save to CSV → Live Table Update → View Stats/Search.

6. Advantages:

- Modern, intuitive UI.
- Validation prevents duplicate records.
- No manual grading or stat calculation required.
- Modular code allows easy addition of new features.

7. Limitations:

While the system is highly functional, it currently operates with the following constraints:

- Flat-File Storage (CSV Only)
- Single-User Desktop System
- Manual File Management
- Static Reporting

8. Future Improvements:

The modular OOP structure of this project allows for significant future expansion:

- Database Migration (SQLite/MySQL)
- PDF Report Generation
- Multimedia Integration
- Visual Data Analytics
- Network Capabilities

9. Conclusion:

The Student Management System successfully demonstrates a Python GUI-based desktop application. The system allows users to manage students, courses, and results efficiently. The use of OOP concepts improves code maintainability and scalability. This project provides a strong foundation for developing real-world academic management systems.